RAMDEOBABA UNIVERSITY, NAGPUR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B.Tech. CSE (Artificial Intelligence and Data Science) (AIDS)

SEMESTER V

SI LAB

PROJECT REPORT

CROP YIELD PREDICTION SYSTEM

Submitted by

Student Name	Roll No.	Enrollment No.
Sanchali Chandankhede	20	

Academic Year: 2026–27

CROP YIELD PREDICTION SYSTEM

SI Lab Project Report

1. PROJECT OBJECTIVE

The main purpose of this project is to build a machine-learning based system that estimates crop yield (in hectograms per hectare, hg/ha) from the location, crop type, year, average rainfall, pesticide usage and average temperature. The problem addressed is that crop yield depends on many interacting factors, so a data-driven model trained on historical records can give a quick and consistent estimate. The expected output is a predicted yield value shown through a web-based interface, and the intended use is as a decision-support tool for students, researchers and agricultural planners.

Since crop yield is a continuous numeric quantity, this project is a regression problem. The classification metrics suggested in the report format (accuracy, precision, recall, F1-score, confusion matrix) are therefore replaced by the regression metrics MAE, RMSE and R^2. The objectives are:

- Load and explore the crop yield dataset.
- Preprocess the data (missing values, duplicates, encoding and scaling) and prepare it for model building.
- Use the available agricultural and climate attributes as input features.
- Train multiple regression models: Linear, Ridge, Lasso, Decision Tree and Random Forest.
- Evaluate and compare model performance using MAE, RMSE and R^2.
- Automatically select the best model and save it to disk.
- Develop a Streamlit GUI to accept inputs and display the predicted yield.

2. INTRODUCTION

Agriculture is a primary source of food and livelihood, and crop production varies from year to year depending on climate, soil, inputs and farming practices. Reliable yield estimation supports food-security planning, resource allocation and policy decisions. Traditionally, such estimates rely on field surveys, which are slow and costly. With the availability of historical agricultural and climate data, machine-learning models can learn the relationship between growing conditions and yield and produce quick estimates.

The motivation behind this project is to apply the machine-learning techniques studied in the SI Lab to a practical, socially relevant problem and to deliver the result as an easy-to-use application rather than only as a notebook.

2.1 Problem Statement

To develop a regression-based system that predicts crop yield (hg/ha) for a given country, crop, year, average annual rainfall, pesticide usage and average temperature, by training and

comparing several regression models and deploying the best one through a graphical interface.

2.2 Scope of the Project

The project includes data loading and cleaning, preprocessing through a reusable pipeline, training and comparison of five regression models, automatic selection and saving of the best model, and a Streamlit web interface for prediction. The limitations are: the dataset covers only the years 1990–2013, 101 countries and 10 crops; it contains no soil, irrigation, seed-variety or fertiliser information; rainfall, pesticide and temperature values are country-level yearly averages rather than farm-level measurements; and the model can only predict for countries and crops present in the data. Predictions are estimates, not guaranteed yields.

3. PROJECT METHODOLOGY

The project follows the stages of dataset loading and exploration, preprocessing, feature selection, training of multiple models, performance evaluation, and best-model selection, followed by GUI development.

3.1 Dataset Loading and Exploration

- **Dataset:** Crop Yield Prediction Dataset (file yield_df.csv), compiled from FAO crop statistics and World Bank climate data and distributed on Kaggle (see References).
- **Size:** 28,242 rows and 8 columns (including an index column, "Unnamed: 0", that carries no information). 101 countries, 10 crops, years 1990–2013.
- **Target variable:** hg/ha_yield (renamed Yield), the crop yield in hectograms per hectare.
- **Features:** Area (Country), Item (Crop), Year, average_rain_fall_mm_per_year (Rainfall), pesticides_tonnes (Pesticides) and avg_temp (Temperature).

Table 1 lists the data type of each column and Table 2 gives summary statistics computed from the dataset.

Column	Data type	Description
Area	object (string)	Country name
Item	object (string)	Crop name
Year	int64	Year of record
hg/ha_yield	int64	Yield (target)
average_rain_fall_mm_per_year	float64	Rainfall (mm/year)
pesticides_tonnes	float64	Pesticide use (tonnes)
avg_temp	float64	Average temperature (°C)

Table 1: Dataset columns and data types

Statistic	Year	Yield	Rainfall	Pesticides	Temp.
Mean	2001.54	77,053.33	1,149.06	37,076.91	20.54
Std. dev.	7.05	84,956.61	709.81	59,958.78	6.31
Minimum	1990	50	51	0.04	1.30
Median	2001	38,295	1,083	17,529.44	21.51
Maximum	2013	501,412	3,240	367,778	30.65

Table 2: Summary statistics of numeric columns

Missing values and duplicates: the dataset has no missing values in any column. No fully duplicate rows exist when the index column is included. However, 2,310 rows are repeated when only the feature and target columns are compared; this is discussed in Section 6.2. The number of crop records per crop ranges from 4,276 (Potatoes) and 4,121 (Maize) to 556 (Plantains and others), so the classes are not equally represented.

Exploratory visualizations are shown below. Figure 1 shows that yield is strongly right-skewed. Figure 2 shows that mean yield differs greatly between crops, with root and tuber crops such as potatoes, cassava and sweet potatoes having the highest averages. Figure 3 shows that the numeric inputs have only weak linear correlation with yield (for example, Temperature −0.12, Year 0.09, Pesticides 0.06, Rainfall 0.00), which suggests that the relationship is mostly non-linear and depends on the categorical features.

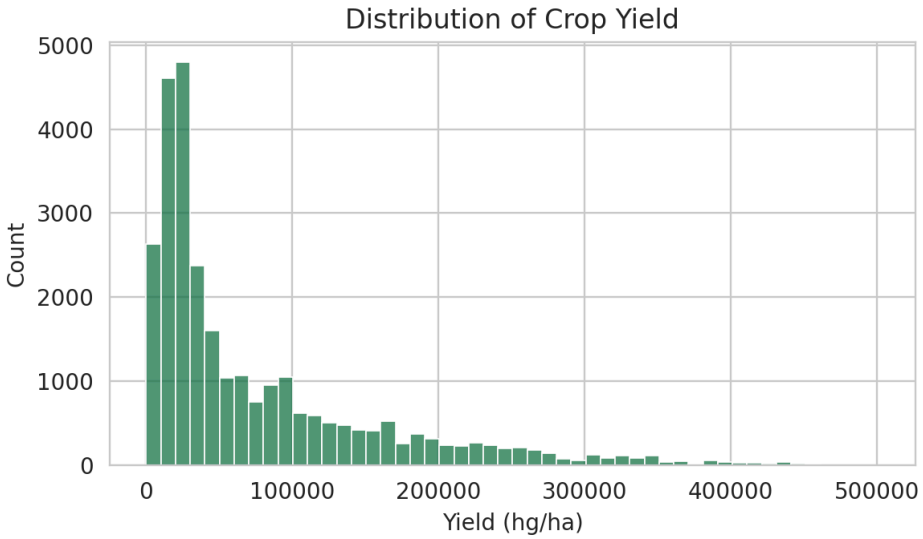


Figure 1: Distribution of crop yield (hg/ha)

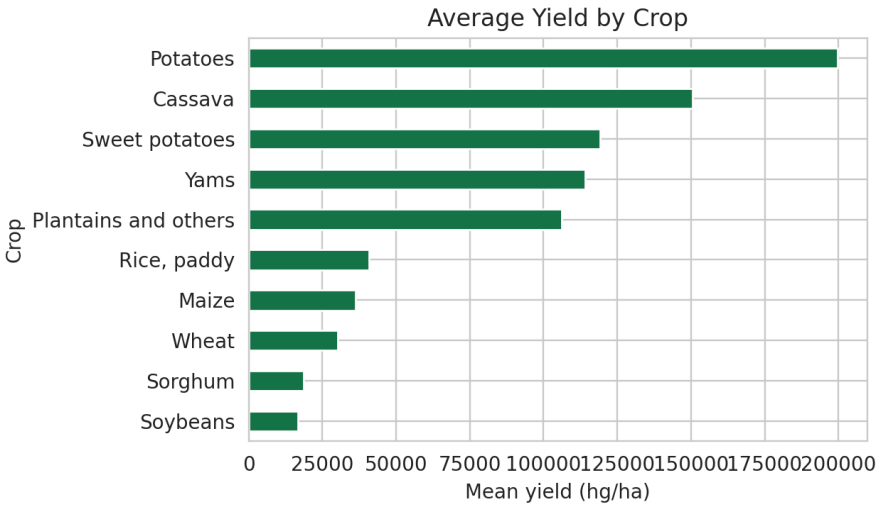


Figure 2: Average yield by crop

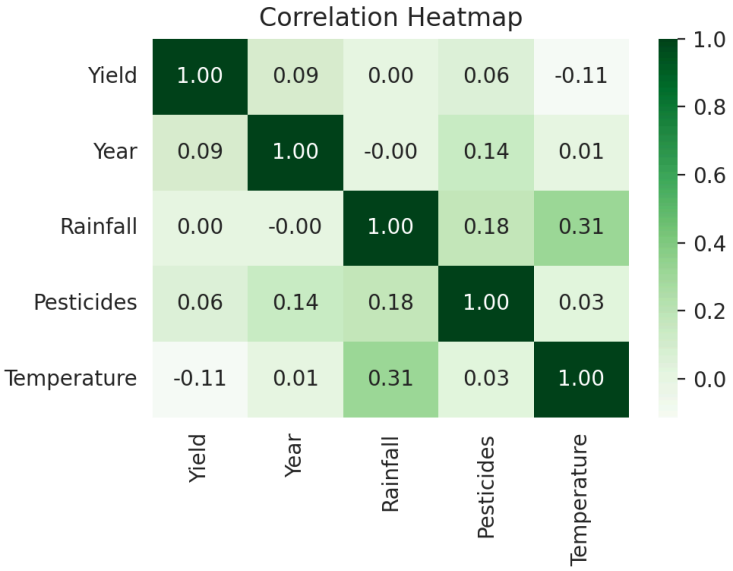


Figure 3: Correlation heatmap of numeric variables and yield

3.2 Data Preprocessing

- Columns were renamed to simpler names (Area→Country, Item→Crop, hg/ha_yield→Yield, average_rain_fall_mm_per_year→Rainfall, pesticides_tonnes→Pesticides, avg_temp→Temperature).
- Numeric columns were converted with pd.to_numeric (invalid values to NaN) and rows with missing values in any feature or the target were dropped; exact duplicate rows were removed. After this step 28,242 records remained.
- Numeric features (Year, Rainfall, Pesticides, Temperature) were passed through a median SimpleImputer and then standardised with StandardScaler.

- Categorical features (Country, Crop) were passed through a most-frequent SimpleImputer and then converted with OneHotEncoder(handle_unknown="ignore"), producing 115 input columns in total (4 numeric + 101 country + 10 crop).
- Input features X (6 columns) were separated from the target y (Yield).
- The data was split into 80 % training (22,593 rows) and 20 % testing (5,649 rows) with random_state = 42.

All these steps are wrapped in a scikit-learn ColumnTransformer inside a Pipeline, so the same preprocessing is automatically applied to new inputs at prediction time and no separate scaler or encoder file is needed.

3.3 Feature Extraction / Selection

No separate feature-extraction technique was required because the dataset already provides meaningful, ready-to-use attributes. All six available attributes (Country, Crop, Year, Rainfall, Pesticides, Temperature) were used as input features; the index column was not used. Categorical attributes are represented by one-hot encoding and numeric attributes by standardised values. The importance of each feature, computed from the final Random Forest model (with one-hot columns of a variable summed together), is shown in Figure 4: Crop (61.2 %) and Country (20.0 %) are the most influential, followed by Pesticides (7.0 %), Rainfall (4.3 %), Temperature (4.2 %) and Year (3.3 %).

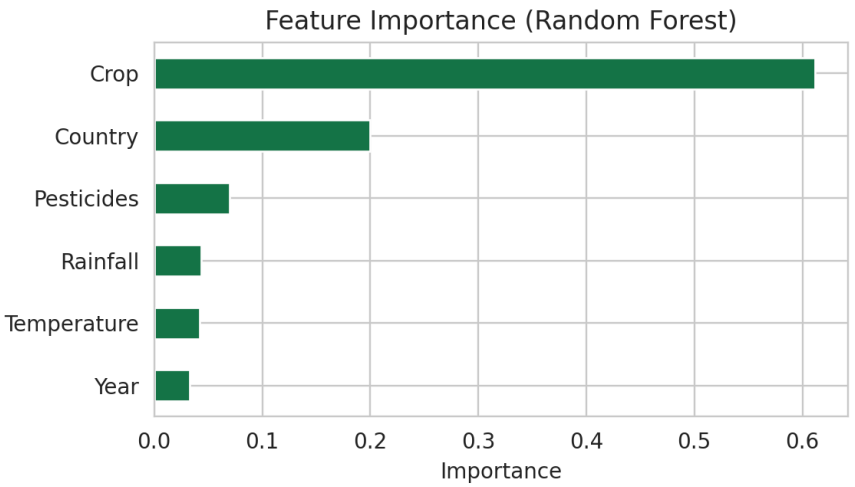


Figure 4: Feature importance from the Random Forest model

3.4 Training Multiple Regression Models

Because the target is numeric, five regression models were trained on the same training set using identical preprocessing.

Sr. No.	Model Name	Purpose / Notes
1	Linear Regression	Baseline model that fits a linear relationship between the inputs and yield.

Sr. No.	Model Name	Purpose / Notes
2	Ridge Regression (alpha = 1.0)	Linear model with L2 regularisation to reduce overfitting and handle correlated one-hot features.
3	Lasso Regression (alpha = 0.01)	Linear model with L1 regularisation that can shrink weak coefficients towards zero.
4	Decision Tree (max_depth = 20)	Non-linear model that splits the data on feature thresholds; captures interactions.
5	Random Forest (150 trees)	Ensemble of decision trees that averages their predictions; robust and accurate on non-linear data.

Table 3: Regression models implemented

3.5 Performance Evaluation and Model Comparison

Since this is a regression task, the models were evaluated on the 20 % test set using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and the R^2 score. Lower MAE and RMSE and higher R^2 indicate a better model. A confusion matrix, accuracy, precision, recall and F1-score apply only to classification and are therefore not used; instead, an actual-versus-predicted plot is provided (Figure 9).

Model	MAE (hg/ha)	RMSE (hg/ha)	R^2 Score
Random Forest	3476.63	9438.07	0.9877
Decision Tree	4628.91	12586.50	0.9782
Ridge Regression	29542.16	42142.67	0.7552
Lasso Regression	29582.28	42144.01	0.7551
Linear Regression	29582.50	42144.04	0.7551

Table 4: Model comparison on the test set (sorted by RMSE)

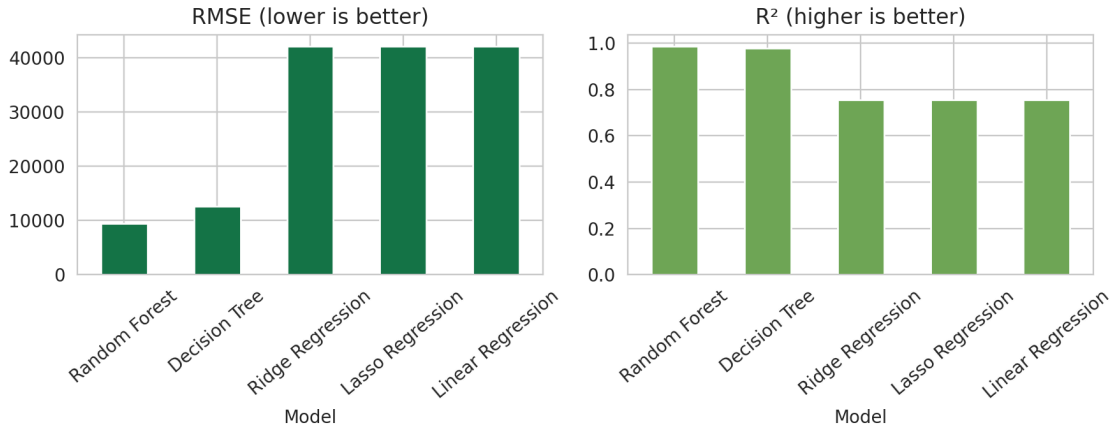


Figure 5: Comparison of RMSE and R^2 across models

3.6 Best Model Selection

The application ranks the models by test-set RMSE and automatically selects the one with the lowest value. Random Forest was selected, with MAE = 3,476.63 hg/ha, RMSE = 9,438.07

```
best_model_name = results_df.loc[
    results_df["RMSE"].idxmin(), "Model"
]

best_model = trained_models[best_model_name]

print("Best Model:", best_model_name)

best_prediction = best_model.predict(X_test)

print("Best Model MAE:",
      mean_absolute_error(y_test, best_prediction))

print("Best Model RMSE:",
      np.sqrt(mean_squared_error(y_test, best_prediction)))

print("Best Model R2 Score:",
      r2_score(y_test, best_prediction))
```

```
Best Model: Random Forest
Best Model MAE: 4004.295105070368
Best Model RMSE: 10556.667945248764
Best Model R2 Score: 0.9846240461581239
```

hg/ha and $R^2 = 0.9877$. It reduces the RMSE by about 25 % compared with the Decision Tree (12,586.50) and by about 78 % compared with Linear Regression (42,144.04). The three linear models perform almost identically ($R^2 \approx 0.755$), which shows that the relationship between the inputs and yield is non-linear. Random Forest is the best choice because it handles non-linearity and the interaction between country and crop, and averaging many trees makes it less prone to overfitting than a single tree. The selected pipeline (preprocessing + model) is saved as best_model.pkl using joblib.

4. PROJECT IMPLEMENTATION

4.1 Tools and Technologies

Tool / Technology	Version	Purpose
Python	3.12	Programming language
Google Colab / VS Code	—	Development environment
Pandas / NumPy	3.0.2 / 2.4.4	Data handling
Scikit-learn	1.8.0	Preprocessing and models
Matplotlib / Seaborn	3.10.8 / 0.13.2	Visualization
Joblib	1.5.3	Saving the trained model
Streamlit	1.65.0	GUI / web interface

Table 5: Tools and technologies

4.2 Module-wise Implementation

- **Dataset loading and exploration module:** load_data() reads yield_df.csv, renames the columns, checks that all required columns exist, converts numeric columns, drops missing values and duplicates and is cached with st.cache_data.

- **Data preprocessing module:** a ColumnTransformer with a numeric pipeline (median imputation and standard scaling) and a categorical pipeline (most-frequent imputation and one-hot encoding).
- **Feature extraction/selection module:** the lists NUMERIC_FEATURES and CATEGORICAL_FEATURES define the six input features; no extra extraction was needed.
- **Model training and evaluation module:** train_best_model() performs the 80/20 split, trains the five pipelines, computes MAE, RMSE and R^2, and ranks the models by RMSE.
- **Best model saving/loading module:** the best pipeline is saved with joblib.dump() to best_model.pkl and kept in memory with st.cache_resource so that it is trained only once per session.
- **GUI input and prediction module:** sidebar input widgets and a Predict button; the inputs are converted into a one-row DataFrame, passed to the best model and the result is displayed.

4.3 GUI Implementation

The interface is built with Streamlit using a wide layout. A green banner at the top shows the project title and a short description. The left sidebar ("Crop Conditions") accepts the inputs: Country / Area (drop-down, default India), Crop Type (drop-down, default Maize), Year (slider from 1990 to 2013), Average Rainfall in mm/year (number input), Pesticides Used in tonnes (number input) and Average Temperature in °C (number input). The number inputs are initialised with the dataset medians.

The main area has two columns. The left column, "Selected Conditions", shows the chosen values as metric cards. The right column, "Prediction Result", initially shows a "Ready" card with the name of the selected model. When the user clicks the Predict Crop Yield button, the inputs are converted into a DataFrame with the same columns used in training and passed to the saved Random Forest pipeline. The pipeline applies imputation, scaling and encoding and returns the predicted yield, which is checked to be finite, clipped at zero and displayed in hg/ha with a confirmation message and a note that it is only an estimate. The footer shows which model was selected and the basis for selection (test-set RMSE). The GUI is started with the command: streamlit run app.py.

As an example, for India, Maize, year 2013 and the dataset median values for rainfall (1,083 mm), pesticides (17,529.44 tonnes) and temperature (21.51 °C), the model gives an estimated yield of about 23,615.85 hg/ha.

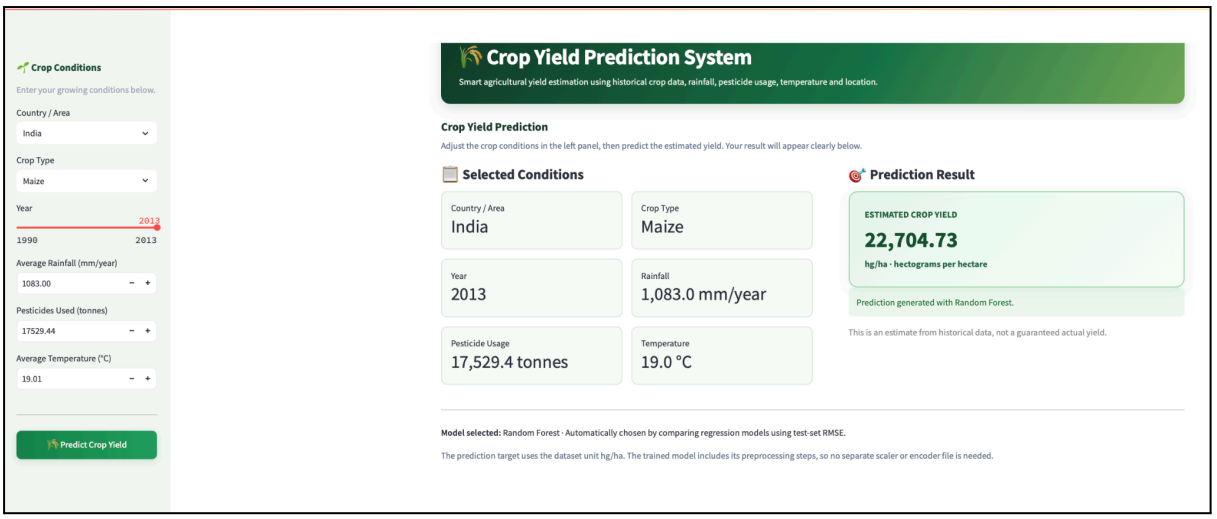


Figure 6: GUI interface

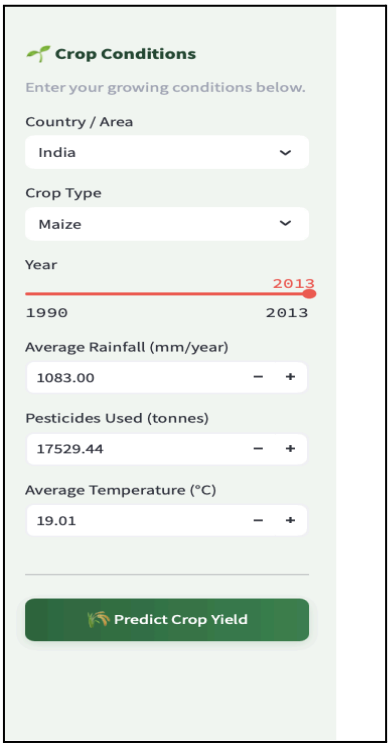


Figure 7: GUI prediction result

5. COMPLETE CODE

The complete source code of the application (app.py) is given below. It contains data loading, preprocessing, training of all models, best-model selection and saving, and the GUI. It is run with "streamlit run app.py" with yield_df.csv in the same folder.

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

# -------------------- PAGE CONFIG --------------------
st.set_page_config(
    page_title="Crop Yield Prediction System",
    page_icon="🌾",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom styling: larger, clearer title and an attractive, responsive layout.
st.markdown("""
<style>
    /* Use more of the available browser width */
    .block-container {
        max-width: 1600px;
        padding-top: 1.2rem;
        padding-bottom: 2.5rem;
        padding-left: 3rem;
        padding-right: 3rem;
    }

    /* Keep the complete project title visible */
    .hero {
        background: linear-gradient(120deg, #123d2a 0%, #167347 58%, #70a957 100%);
        padding: 1.65rem 2rem;
        border-radius: 18px;
        color: #ffffff;
        margin-bottom: 1.25rem;
        box-shadow: 0 8px 24px rgba(18, 61, 42, 0.16);
    }
    .hero-title {
        font-size: clamp(2rem, 3vw, 3.15rem);
        line-height: 1.2;
        font-weight: 850;
        letter-spacing: -0.8px;
        color: #ffffff;
        margin: 0;
        overflow-wrap: anywhere;
    }
    .hero-subtitle {
        font-size: 1.05rem;
        line-height: 1.6;
        margin-top: 0.6rem;
        color: #e8f5e9;
    }

    h2, h3 {font-weight: 750 !important;}
    [data-testid="stSidebar"] {
        background: #f1f6f2;
    }
    [data-testid="stSidebar"] h2 {
        color: #174b32;
        font-weight: 800;
    }
    [data-testid="stSidebar"] .block-container {
        padding-top: 1.5rem;
    }

    div.stButton > button {
        width: 100%;
        min-height: 3.2rem;
        background: linear-gradient(90deg, #16834b, #20a05d);
        color: white;
        border: 0;
        border-radius: 10px;
        padding: 0.75rem 1rem;
        font-size: 1.05rem;
        font-weight: 750;
        box-shadow: 0 4px 12px rgba(22, 131, 75, 0.18);
    }
    div.stButton > button:hover {
        background: linear-gradient(90deg, #11683b, #16834b);
        color: white;
        border: 0;
    }

    .section-heading {
        font-size: 1.45rem;
        font-weight: 800;
        color: #193b2b;
        margin: 0.3rem 0 0.35rem 0;
    }
```

```
    .section-caption {
        color: #64748b;
        font-size: 1rem;
        line-height: 1.6;
        margin-bottom: 1rem;
    }

    .result-card {
        background: linear-gradient(135deg, #e8f8ed 0%, #f8fff9 100%);
        border: 2px solid #9bd8ad;
        border-radius: 18px;
        padding: 1.7rem;
        min-height: 190px;
        box-shadow: 0 8px 22px rgba(21, 128, 61, 0.08);
    }
    .result-label {
        color: #22643d;
        font-size: 1.05rem;
        font-weight: 750;
    }
    .result-value {
        color: #14532d;
        font-size: clamp(2.3rem, 3vw, 3.3rem);
        line-height: 1.2;
        font-weight: 850;
        margin: 0.55rem 0;
        overflow-wrap: anywhere;
    }
    .result-unit {
        color: #276749;
        font-size: 1.05rem;
        font-weight: 750;
    }

    div[data-testid="stMetric"] {
        background: #f7faf8;
        border: 1px solid #e0ebe3;
        padding: 1rem;
        border-radius: 12px;
        min-height: 105px;
    }
    div[data-testid="stMetricLabel"] {
        font-weight: 650;
    }
    .note {
        color: #64748b;
        font-size: 0.9rem;
        line-height: 1.55;
    }
    hr {margin: 1.7rem 0;}
    @media (max-width: 800px) {
        .block-container {
            padding-left: 1rem;
            padding-right: 1rem;
        }
        .hero {padding: 1.25rem;}
    }
</style>
""", unsafe_allow_html=True)

DATA_FILE = "yield_df.csv"
MODEL_FILE = "best_model.pkl"

RENAME_COLUMNS = {
    "Area": "Country",
    "Item": "Crop",
    "hg/ha_yield": "Yield",
    "average_rain_fall_mm_per_year": "Rainfall",
    "pesticides_tonnes": "Pesticides",
    "avg_temp": "Temperature"
}

NUMERIC_FEATURES = ["Year", "Rainfall", "Pesticides", "Temperature"]
CATEGORICAL_FEATURES = ["Country", "Crop"]
FEATURES = CATEGORICAL_FEATURES + NUMERIC_FEATURES
TARGET = "Yield"


@st.cache_data
def load_data(file_path):
    df = pd.read_csv(file_path).rename(columns=RENAME_COLUMNS)
    required = FEATURES + [TARGET]
    missing = [col for col in required if col not in df.columns]
    if missing:
        raise ValueError(
```

```
            "Dataset columns missing: " + ", ".join(missing) +
            ". Use the original crop yield dataset with Area, Item, Year, "
            "hg/ha_yield, average_rain_fall_mm_per_year, pesticides_tonnes and avg_temp."
        )

    for col in NUMERIC_FEATURES + [TARGET]:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    df = df.dropna(subset=FEATURES + [TARGET]).drop_duplicates()
    df["Country"] = df["Country"].astype(str)
    df["Crop"] = df["Crop"].astype(str)
    if df.empty:
        raise ValueError("No valid records were found in the dataset.")
    return df


@st.cache_resource
def train_best_model(data):
    X, y = data[FEATURES], data[TARGET]

    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ])
    preprocessor = ColumnTransformer([
        ("numeric", numeric_pipeline, NUMERIC_FEATURES),
        ("categorical", categorical_pipeline, CATEGORICAL_FEATURES)
    ])

    models = {
        "Linear Regression": LinearRegression(),
        "Ridge Regression": Ridge(alpha=1.0),
        "Lasso Regression": Lasso(alpha=0.01, max_iter=10000),
        "Decision Tree": DecisionTreeRegressor(random_state=42, max_depth=20),
        "Random Forest": RandomForestRegressor(
            n_estimators=150, random_state=42, n_jobs=-1
        )
    }

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    rows, fitted = [], {}

    for name, regressor in models.items():
        pipeline = Pipeline([
            ("preprocessor", preprocessor),
            ("model", regressor)
        ])
        pipeline.fit(X_train, y_train)
        predictions = pipeline.predict(X_test)
        rows.append({
            "Model": name,
            "MAE": float(mean_absolute_error(y_test, predictions)),
            "RMSE": float(np.sqrt(mean_squared_error(y_test, predictions))),
            "R² Score": float(r2_score(y_test, predictions))
        })
        fitted[name] = pipeline

    comparison = pd.DataFrame(rows).sort_values("RMSE").reset_index(drop=True)
    best_name = comparison.loc[0, "Model"]
    best_model = fitted[best_name]
    joblib.dump(best_model, MODEL_FILE)
    return best_model, best_name, comparison


# -------------------- HEADER --------------------
st.markdown("""
<div class="hero">
    <div class="hero-title">🌾 Crop Yield Prediction System</div>
    <div class="hero-subtitle">
        Smart agricultural yield estimation using historical crop data,
        rainfall, pesticide usage, temperature and location.
    </div>
</div>
""", unsafe_allow_html=True)

try:
    data = load_data(DATA_FILE)
except FileNotFoundError:
    st.error("Dataset not found. Place **yield_df.csv** in the same folder as **app.py**.")
```

```
    st.stop()
except Exception as exc:
    st.error(f"Unable to read the dataset: {exc}")
    st.stop()

with st.spinner("Preparing the crop yield prediction model..."):
    try:
        best_model, best_model_name, comparison = train_best_model(data)
    except Exception as exc:
        st.error(f"Model training failed: {exc}")
        st.stop()

countries = sorted(data["Country"].unique().tolist())
crops = sorted(data["Crop"].unique().tolist())

# -------------------- SIDEBAR INPUTS --------------------
with st.sidebar:
    st.markdown("## 🌱 Crop Conditions")
    st.caption("Enter your growing conditions below.")

    country = st.selectbox(
        "Country / Area", countries,
        index=countries.index("India") if "India" in countries else 0
    )
    crop = st.selectbox(
        "Crop Type", crops,
        index=crops.index("Maize") if "Maize" in crops else 0
    )

    year_min, year_max = int(data["Year"].min()), int(data["Year"].max())
    year_default = min(max(2013, year_min), year_max)
    year = st.slider("Year", year_min, year_max, year_default)

    rainfall_median = float(data["Rainfall"].median())
    rainfall_max = max(float(data["Rainfall"].max()) * 2, 1.0)
    rainfall = st.number_input(
        "Average Rainfall (mm/year)",
        min_value=0.0, max_value=rainfall_max,
        value=min(max(rainfall_median, 0.0), rainfall_max),
        step=10.0
    )

    pesticide_median = max(float(data["Pesticides"].median()), 0.0)
    pesticide_max = max(float(data["Pesticides"].max()) * 2, 1.0)
    pesticides = st.number_input(
        "Pesticides Used (tonnes)",
        min_value=0.0, max_value=pesticide_max,
        value=min(pesticide_median, pesticide_max),
        step=10.0
    )

    temp_min = min(-20.0, float(data["Temperature"].min()) - 5)
    temp_max = max(60.0, float(data["Temperature"].max()) + 5)
    temperature = st.number_input(
        "Average Temperature (°C)",
        min_value=temp_min, max_value=temp_max,
        value=float(data["Temperature"].median()),
        step=0.5
    )

    st.markdown("---")
    predict_clicked = st.button("🌾 Predict Crop Yield", use_container_width=True)

# -------------------- MAIN PREDICTION AREA --------------------
st.markdown('<div class="section-heading">Crop Yield Prediction</div>',
unsafe_allow_html=True)
st.markdown(
    '<div class="section-caption">Adjust the crop conditions in the left panel, then
predict '
    'the estimated yield. Your result will appear clearly below.</div>',
    unsafe_allow_html=True
)

condition_col, result_col = st.columns([1.1, 1], gap="large")

with condition_col:
    st.markdown("### 📋 Selected Conditions")
    a, b = st.columns(2)
    a.metric("Country / Area", country)
    b.metric("Crop Type", crop)
    a.metric("Year", str(year))
    b.metric("Rainfall", f"{rainfall:,.1f} mm/year")
    a.metric("Pesticide Usage", f"{pesticides:,.1f} tonnes")
    b.metric("Temperature", f"{temperature:,.1f} °C")
```

```
with result_col:
    st.markdown("### 🎯 Prediction Result")
    if predict_clicked:
        input_df = pd.DataFrame([{
            "Country": country,
            "Crop": crop,
            "Year": year,
            "Rainfall": rainfall,
            "Pesticides": pesticides,
            "Temperature": temperature
        }], columns=FEATURES)

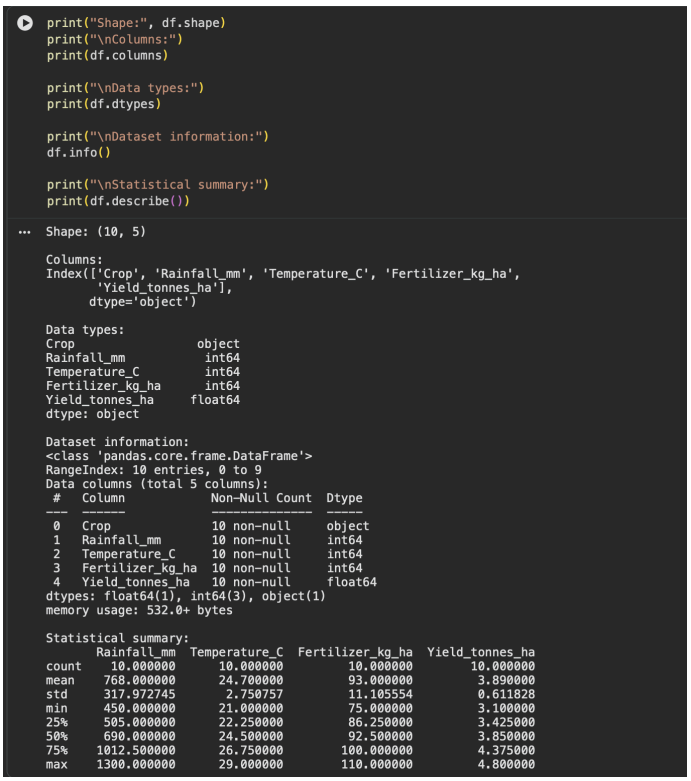
        try:
            predicted_yield = float(best_model.predict(input_df)[0])
            if not np.isfinite(predicted_yield):
                raise ValueError("The model returned an invalid result.")
            predicted_yield = max(0.0, predicted_yield)
            st.markdown(
                f'<div class="result-card">'
                f'<div class="result-label">ESTIMATED CROP YIELD</div>'
                f'<div class="result-value">{predicted_yield:,.2f}</div>'
                f'<div class="result-unit">hg/ha · hectograms per hectare</div>'
                f'</div>',
                unsafe_allow_html=True
            )
            st.success(f"Prediction generated with {best_model_name}.")
            st.caption("This is an estimate from historical data, not a guaranteed actual
yield.")
        except Exception as exc:
            st.error(f"Prediction failed: {exc}")
    else:
        st.info("Enter your crop conditions and click **Predict Crop Yield**.")
        st.markdown(
            f'<div class="result-card">'
            f'<div class="result-label">YOUR RESULT WILL APPEAR HERE</div>'
            f'<div class="result-value">🌱 Ready</div>'
            f'<div class="result-unit">Selected model: {best_model_name}</div>'
            f'</div>',
            unsafe_allow_html=True
        )

st.markdown("---")
st.markdown(
    f"**Model selected:** {best_model_name}  ·  "
    "Automatically chosen by comparing regression models using test-set RMSE."
)
st.markdown(
    '<div class="note">The prediction target uses the dataset unit hg/ha. '
    'The trained model includes its preprocessing steps, so no separate scaler or encoder
file is needed.</div>',
    unsafe_allow_html=True
)
```

6. RESULTS AND DISCUSSION

6.1 Output Screenshots and Results

Data preprocessing output: after loading, renaming and cleaning, the dataset has 28,242 rows with 0 missing values. After the train-test split the training set has 22,593 rows and the test set has 5,649 rows. One-hot encoding of Country and Crop together with the four numeric features gives 115 columns for the models.



```
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns)

print("\nData types:")
print(df.dtypes)

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
print(df.describe())
```

```
Shape: (10, 5)

Columns:
Index(['Crop', 'Rainfall_mm', 'Temperature_C', 'Fertilizer_kg_ha',
       'Yield_tonnes_ha'],
      dtype='object')

Data types:
Crop                object
Rainfall_mm          int64
Temperature_C        int64
Fertilizer_kg_ha     int64
Yield_tonnes_ha    float64
dtype: object

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Crop              10 non-null     object
 1   Rainfall_mm       10 non-null     int64
 2   Temperature_C     10 non-null     int64
 3   Fertilizer_kg_ha  10 non-null     int64
 4   Yield_tonnes_ha   10 non-null     float64
dtypes: float64(1), int64(3), object(1)
memory usage: 532.0+ bytes

Statistical summary:
       Rainfall_mm  Temperature_C  Fertilizer_kg_ha  Yield_tonnes_ha
count    10.000000      10.000000         10.000000        10.000000
mean    768.000000      24.700000         93.000000         3.890000
std     317.972745       2.750757         11.105554         0.611828
min     450.000000      21.000000         75.000000         3.100000
25%     505.000000      22.250000         86.250000         3.425000
50%     690.000000      24.500000         92.500000         3.850000
75%    1012.500000      26.750000        100.000000         4.375000
max    1300.000000      29.000000        110.000000         4.800000
```

```
print("Missing values:")
print(df.isnull().sum())

print("Duplicate records:", df.duplicated().sum())

df = df.drop_duplicates()
```

```
Missing values:
Crop               0
Rainfall_mm        0
Temperature_C      0
Fertilizer_kg_ha   0
Yield_tonnes_ha    0
dtype: int64
Duplicate records: 0
```

Step 4: Check unique values and frequency

```
print("Unique crops:")
print(df["Crop"].unique())

print("\nNumber of crop types:")
print(df["Crop"].nunique())

print("\nRecords per crop:")
print(df["Crop"].value_counts())
```

```
Unique crops:
['Wheat' 'Rice' 'Maize']

Number of crop types:
3

Records per crop:
Crop
Wheat    4
Rice     3
Maize    3
Name: count, dtype: int64
```

Figure 8: Data preprocessing output

Model comparison and performance results: Table 4 and Figure 5 present the MAE, RMSE and R^2 of all five models. Random Forest is first, followed by Decision Tree, with the three linear models far behind.

Best model selection: Random Forest was selected automatically (lowest RMSE = 9,438.07). The application displays “Model selected: Random Forest” in its footer. Figure 9 compares actual and predicted yield for the test set; the points lie close to the red diagonal line, which indicates accurate predictions.

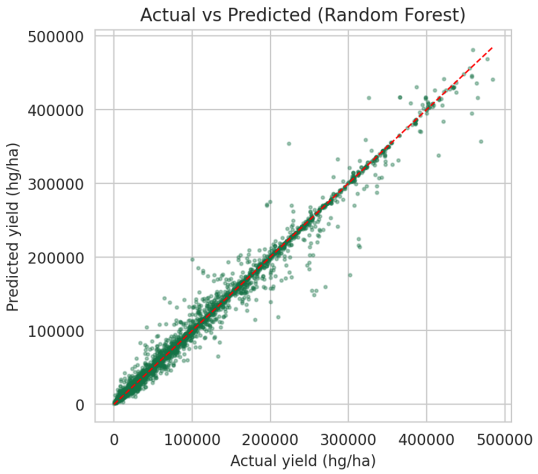


Figure 9: Actual vs predicted yield for the Random Forest model (test set)

GUI interface and prediction results: see Figures 6 and 7 in Section 4.3.

6.2 Result Analysis

- **Model differences:** The linear models ($R^2 \approx 0.755$, RMSE ≈ 42,140) cannot capture the non-linear influence of crop and country. The Decision Tree improves this considerably ($R^2 = 0.978$) and the Random Forest is best ($R^2 = 0.988$, MAE = 3,476.63 hg/ha, about 4.5 % of the mean yield of 77,053 hg/ha).
- **Strengths:** high accuracy on unseen test data, a single pipeline that includes preprocessing, automatic model selection, and a simple interface that does not require any machine-learning knowledge.
- **Errors:** an RMSE (9,438 hg/ha) that is noticeably larger than the MAE (3,477 hg/ha) shows that a small number of high-yield records produce large errors. Errors are expected to be larger for crops with very high yields, such as potatoes and cassava.
- **Possible duplicate effect:** The cleaning step removes duplicates with the index column still present, so 2,310 rows that are identical in their features and yield were not removed. Such rows can appear in both the training and test sets and may make the scores slightly optimistic. As a check, the Random Forest was re-evaluated after removing these rows (25,932 records) and still gave $R^2 = 0.984$, RMSE = 10,658.71 and MAE = 4,017.89, so the conclusion that Random Forest is the best model does not change.
- **Limitations:** The data ends in 2013; the model uses country-level averages and ignores soil, irrigation, fertiliser and seed variety; a random split means the test data comes from the same countries and years as the training data, so performance on

completely new regions or future years may be lower; the app cannot predict for countries or crops not in the dataset; and the Random Forest has no hyperparameter tuning or cross-validation.

- **Objectives:** All objectives were achieved: the data was explored and preprocessed, five regression models were trained and compared, the best model was selected and saved, and a working GUI was developed.

7. CONCLUSION

This project developed a Crop Yield Prediction System using the historical crop yield dataset of 28,242 records covering 101 countries and 10 crops. After preprocessing with imputation, scaling and one-hot encoding, five regression models (Linear, Ridge, Lasso, Decision Tree and Random Forest) were trained and compared using MAE, RMSE and R^2. The Random Forest achieved the best results (MAE = 3,476.63 hg/ha, RMSE = 9,438.07 hg/ha, R^2 = 0.9877) and was selected as the final model and saved as best_model.pkl. A Streamlit application allows a user to enter the country, crop, year, rainfall, pesticide usage and temperature and obtain the predicted yield.

The main limitations are the lack of soil, irrigation and other farm-level data, the 1990–2013 time range, and the use of a single random train-test split. Possible improvements include removing duplicates after dropping the index column, hyperparameter tuning and cross-validation, a time- or country-based split for a stricter evaluation, trying gradient-boosting models (XGBoost, LightGBM), adding soil, fertiliser and more recent weather data, and deploying the application online.

8. GITHUB LINK

GitHub Repository URL:

https://github.com/Sanchalichandankhede/crop-prediction-project.git

The repository should contain app.py, yield_df.csv, the trained model file best_model.pkl, the Colab notebook (.ipynb) and its PDF export.

9. REFERENCES

[1] Crop Yield Prediction Dataset (yield_df.csv; data from FAO and World Bank), Kaggle. https://www.kaggle.com/datasets/patelris/crop-yield-prediction-dataset

[2] Scikit-learn documentation. https://scikit-learn.org/stable/

[3] Streamlit documentation. https://docs.streamlit.io/

[4] Pandas documentation. https://pandas.pydata.org/docs/

[5] Joblib documentation. https://joblib.readthedocs.io/

[6] Matplotlib and Seaborn documentation. https://matplotlib.org/ and https://seaborn.pydata.org/